Introduction

ggplot2 is built on Wilkinson's grammar of graphics, which separates two concerns that ad-hoc plotting languages tend to conflate. An **aesthetic** is a mapping from a column of data to a visual property: x-position, y-position, colour, fill, shape, size. A **geom** is the geometric object drawn using those aesthetics: a point, a line, a bar, a histogram, a smoother. Once you internalise the split — "what variable becomes which visual property" versus "what shape do I want drawn" — ggplot2 becomes a Lego kit of swappable parts rather than a dialect to memorise.

Prerequisites

A working knowledge of basic ggplot2 syntax (`ggplot()`, `aes()`, `+ geom_*`), and tidy data conventions where each row is an observation and each column a variable.

Theory

An **aesthetic mapping** lives inside `aes()` and binds a column to a visual property:

```
aes(x = flipper_length_mm, y = body_mass_g, colour = species)
```

The mapping triggers automatic scale construction, legend generation, and axis labelling — ggplot2 knows that a continuous numeric column needs a continuous scale and a categorical column needs a discrete one.

A **geom** consumes the aesthetics and draws the corresponding marks. Common geoms include `geom_point()` for scatter, `geom_line()` for lines, `geom_bar()` and `geom_col()` for bars, `geom_histogram()` and `geom_density()` for distributions, `geom_smooth()` for trend lines.

Mapping vs setting is the most important distinction in ggplot2. Inside `aes()` binds a visual property to a variable, producing one value per row and a legend. Outside `aes()` sets a constant for all rows with no legend. `geom_point(colour = "blue")` makes every point blue; `geom_point(aes(colour = species))` colours points by species.

Multiple geoms in the same plot share base aesthetics declared in `ggplot(aes(...))` and may add their own. This layering is the heart of the grammar: a complex figure is just a stack of compatible geoms reading the same data.

Assumptions

Tidy data: one observation per row, one variable per column. Long-format frames are the standard input for ggplot2; wide-format data should be reshaped via `tidyr::pivot_longer()` before plotting.

R Implementation

```
library(ggplot2); library(palmerpenguins)
data(penguins, package = "palmerpenguins")

# Basic mapping: x, y, colour
ggplot(penguins, aes(flipper_length_mm, body_mass_g, colour = species)) +
  geom_point(alpha = 0.7) +
  theme_minimal()

# Adding a smoother geom on top
ggplot(penguins, aes(flipper_length_mm, body_mass_g, colour = species)) +
  geom_point(alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal()
```

```r
# Aesthetics differ between geoms: shape is mapped only for points
ggplot(penguins, aes(flipper_length_mm, body_mass_g)) +
  geom_point(aes(colour = species, shape = sex), alpha = 0.7) +
  theme_minimal()

# Constant aesthetics outside aes()
ggplot(penguins, aes(flipper_length_mm, body_mass_g)) +
  geom_point(colour = "steelblue", size = 3, alpha = 0.6) +
  theme_minimal()
```

Output & Results

Four versions of the same scatter: a basic colour-coded plot, the same plot with per-species linear smoothers added, a four-aesthetic plot mapping species to colour and sex to shape simultaneously, and a single-colour version where every aesthetic except position is held constant. Each illustrates a different combination of mapping and setting.

Interpretation

A reporting sentence (figure caption): "Body mass against flipper length (n = 333 penguins, three species); colour encodes species and shape encodes sex. Per-species linear smoothers are overlaid in the right panel." Be explicit about which aesthetics carry information and which are constant; readers should not have to infer the mapping.

Practical Tips

- Put aesthetics shared across geoms in the base `ggplot()` call; put geom-specific aesthetics inside the geom. Repetition is fine for clarity but is rarely necessary.
- Reach for `alpha` to mitigate overplotting in dense scatter plots; values between 0.2 and 0.5 typically reveal density structure.
- `fill` controls the interior of geoms with area (bars, boxplots, polygons); `colour` controls edges and points. Mixing them up is one of the most common `ggplot2` errors.
- For categorical colour scales, prefer `viridis::scale_colour_viridis_d()` or `RColorBrewer::scale_colour_brewer()` avoid the default rainbow.
- Use `shape` only for two to six groups; beyond that, shapes become indistinguishable and a facet or text label communicates better.
- When you want a constant value but `ggplot2` keeps drawing a legend, you forgot to move the assignment outside `aes()`; this is the single most common debugging step.

R Packages Used

`ggplot2` for the core grammar of graphics; `palmerpenguins` for a clean teaching dataset rich in aesthetic-mapping opportunities; `viridis` and `RColorBrewer` for accessible categorical and continuous scales; `ggrepel` for non-overlapping point labels that supplement aesthetic mappings with text.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- ggplot2 grammar and multi-panel layouts